



Firmwares para teste de componentes isolados

1. Objetivo

- ! Este documento tem como objetivo elencar os códigos utilizados para a testagem isolada dos componentes individuais do micromouse. Objetiva-se, também, nortear a subequipe de `#firmware` quanto às bibliotecas e comandos utilizados para a leitura e tratamento dos dados de cada um dos sensores e atuadores, para a integração posterior desses dados com o firmware principal, que contém o algoritmo de navegação. Por fim, tendo em vista o ambiente de programação escolhido pela equipe de `#hardware` para o desenvolvimento do firmware (PlatformIO via Arduino IDE), todos os códigos abaixo utilizam a biblioteca Arduino.h

2. Códigos

Foram elaborados códigos para o teste dos seguintes componentes:

1. ESP32-S3;

2. MPU-6050;
3. Sensores ToF VL53L0X;
4. Ponte H L298N (e motores DC 6V);
5. Medidor de carga da bateria.

❗ **Nota 1:** o circuito de alimentação (bateria e regulador de tensão) não será testado via firmware, pois a medição de tensão nos terminais de saída desses componentes, juntamente com testes de consumo e durabilidade de bateria, já são suficientes para validar o funcionamento desses componentes.

❗ **Nota 2:** se o monitor serial não exibir nada mesmo com os delays, adicionar a linha `build_flags = -D ARDUINO_USB_CDC_ON_BOOT=1` no arquivo `platformio.ini`. Isso força o ESP32-S3 a rotear a comunicação Serial para a porta USB correta.

2.1. ESP32-S3

O firmware desenvolvido visa testar o funcionamento do multi-core da ESP32-S3, criando duas tarefas diferentes e alocando cada uma delas a um núcleo do microcontrolador. O resultado é impresso via serial.

```
#include <Arduino.h>

// Tarefa dedicada ao Core 0 (Ideal para leitura rápida de sensores I2C)
void taskCore0(void * pvParameters) {
    for(;;) {
        Serial.print("Ping do Core 0! Processador rodando perfeitamente. ID: ");
        Serial.println(xPortGetCoreID());
        vTaskDelay(pdMS_TO_TICKS(1000)); // Delay não-bloqueante do FreeRTOS
    }
}
```

```

    }
}

// Tarefa dedicada ao Core 1 (Ideal para PID e controle da Ponte H)
void taskCore1(void * pvParameters) {
    for(;;) {
        Serial.print("Pong do Core 1! Processador rodando perfeitamente. ID: ");
        Serial.println(xPortGetCoreID());
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

void setup() {
    Serial.begin(115200);

    // O ESP32-S3 usa Native USB na maioria das placas.
    // Esse delay garante que a porta serial do PC tenha tempo de conectar.
    delay(3000);

    Serial.println("--- Iniciando Teste de Diagnóstico do ESP32-S3 ---");

    // Cria e fixa a tarefa no Core 0
    xTaskCreatePinnedToCore(
        taskCore0,          /* Função que implementa a tarefa */
        "Tarefa_Sensores", /* Nome para debug */
        10000,              /* Tamanho da stack em bytes */
        NULL,               /* Parâmetros passados para a tarefa */
        1,                 /* Prioridade (1 é a padrão) */
        NULL,               /* Handle da tarefa (não usado aqui) */
        0);                 /* Fixar no Core 0 */

    // Cria e fixa a tarefa no Core 1

```

```

xTaskCreatePinnedToCore(
    taskCore1,
    "Tarefa_Motores",
    10000,
    NULL,
    1,
    NULL,
    1);          /* Fixar no Core 1 */
}

void loop() {
    // Como as tarefas gerenciam o processamento, deletamos o loop padrão
    // para liberar totalmente os recursos do Core 1 (onde o loop costuma rodar).
    vTaskDelete(NULL);
}

```

2.2. MPU-6050

```

#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>

// Definição explícita dos pinos I2C para o ESP32-S3
#define SDA_PIN 8
#define SCL_PIN 9

Adafruit_MPU6050 mpu;

void setup(void) {
    Serial.begin(115200);
    delay(3000); // Dá tempo para o USB CDC (Nativo do S3) inicializar no PC
}

```



```

Serial.println("Inicializando MPU6050");

// Inicializa o barramento I2C nos pinos específicos do S3
Wire.begin(SDA_PIN, SCL_PIN);

if (!mpu.begin()) {
    Serial.println("Falha ao encontrar o MPU6050. Verifique as conexões SDA(8) e SCL(9)!");
    while (1) { delay(10); }
}
Serial.println("MPU6050 Encontrado!");

mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
mpu.setGyroRange(MPU6050_RANGE_500_DEG);
mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
}

void loop() {
    sensors_event_t a, g, temp;
    mpu.getEvent(&a, &g, &temp);

    Serial.print("Aceleração X: "); Serial.print(a.acceleration.x);
    Serial.print(", Y: "); Serial.print(a.acceleration.y);
    Serial.print(", Z: "); Serial.print(a.acceleration.z);
    Serial.println(" m/s^2");

    Serial.println("---");
    delay(500);
}

```

2.3. ToF VL53L0X

```

#include <Wire.h>
#include <Adafruit_VL53L0X.h>

```

```

#define SDA_PIN 8
#define SCL_PIN 9

Adafruit_VL53L0X lox = Adafruit_VL53L0X();

void setup() {
  Serial.begin(115200);
  delay(3000);

  Serial.println("Inicializando VL53L0X...");

  Wire.begin(SDA_PIN, SCL_PIN);

  if (!lox.begin()) {
    Serial.println("Falha ao iniciar o VL53L0X. Verifique SDA
(8) e SCL(9).");
    while(1);
  }
  Serial.println("VL53L0X Iniciado com sucesso!");
}

void loop() {
  VL53L0X_RangingMeasurementData_t measure;

  lox.rangingTest(&measure, false);

  if (measure.RangeStatus != 4) {
    Serial.print("Distância: ");
    Serial.print(measure.RangeMilliMeter);
    Serial.println(" mm");
  } else {
    Serial.println("Fora de alcance");
  }
}

```

```
    delay(250);  
}
```

2.4. Ponte H L298N com os dois motores DC

```
#include <Arduino.h>  
  
// Pinos Seguros no ESP32-S3 para o Motor A (Esquerdo)  
const int enA = 4; // PWM  
const int in1 = 5; // Direção  
const int in2 = 6; // Direção  
  
// Pinos Seguros no ESP32-S3 para o Motor B (Direito)  
const int enB = 7; // PWM  
const int in3 = 15; // Direção  
const int in4 = 16; // Direção  
  
const int freq = 5000;  
const int resolution = 8; // 0 a 255  
  
void setup() {  
    pinMode(in1, OUTPUT);  
    pinMode(in2, OUTPUT);  
    pinMode(in3, OUTPUT);  
    pinMode(in4, OUTPUT);  
  
    // NOVA API do ESP32 Core 3.x para configurar o PWM diretamente no pino  
    ledcAttach(enA, freq, resolution);  
    ledcAttach(enB, freq, resolution);  
}  
  
void loop() {  
    // Ajuste a velocidade usando o pino diretamente na nova API  
    I
```

```
    ledcWrite(enA, 180);
    ledcWrite(enB, 180);

    // 1. Motores para FRENTE
    digitalWrite(in1, HIGH);
    digitalWrite(in2, LOW);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
    delay(2000);

    // 2. PARAR Motores
    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, LOW);
    delay(1000);

    // 3. Motores para TRÁS
    digitalWrite(in1, LOW);
    digitalWrite(in2, HIGH);
    digitalWrite(in3, LOW);
    digitalWrite(in4, HIGH);
    delay(2000);

    // 4. PARAR Motores
    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, LOW);
    delay(2000);
}
```

2.5. Medidor de carga da bateria

```

#include <Arduino.h>

// Definir o pino ADC onde o sinal ADC_BAT está conectado.
const int pinoBat = 4;

// Fator de multiplicação calculado a partir dos resistores
(R1=220k, R2=100k)
const float fatorDivisor = 3.2;

void setup() {
  Serial.begin(115200);
  delay(3000); // Tempo para conectar o USB Nativo

  Serial.println("--- teste do medidor de bateria ---");

  // Configura a resolução do ADC para 12 bits (0 a 4095) - P
  padrão do S3
  analogReadResolution(12);

  // A atenuação de 11dB (padrão) permite ler até aprox ~3.1V
  a ~3.3V
  analogSetPinAttenuation(pinoBat, ADC_11db);
}

void loop() {
  // 1. Lê a tensão calibrada diretamente do pino em milivolt
  s
  uint32_t tensaoPino_mV = analogReadMilliVolts(pinoBat);

  // 2. Converte para Volts
  float tensaoPino_V = tensaoPino_mV / 1000.0;

  // 3. Calcula a tensão real da bateria multiplicando pelo f
  ator do divisor
  float tensaoBateria = tensaoPino_V * fatorDivisor;

```

```

// 4. Exibe os resultados
Serial.print("Tensão no Pino (ADC): ");
Serial.print(tensaoPino_V);
Serial.print(" V ---> ");

Serial.print("Tensão da Bateria: ");
Serial.print(tensaoBateria);
Serial.println(" V");

// Um pequeno aviso se a bateria LiPo 2S estiver descarrega
ndo (abaixo de 7.0V)
if (tensaoBateria > 1.0 && tensaoBateria < 7.0) {
    Serial.println("ALERTA: Bateria fraca! Recarregue a LiP
o.");
}

Serial.println("---");
delay(1000);
}

```